UNIDAD 8. Entrada/salida, ficheros y persistencia

Índice

1. Fundamentos de los Flujos (Streams) 2
   1.1. Concepto de Stream: El flujo de datos en Java 2
   1.2. Tipos de flujos: Bytes vs. Caracteres 3
   1.3. Jerarquía de Clases 5
   1.4. Utilización de flujos: Patrones básicos y decoradores 7
2. Entrada y Salida Estándar 10
   2.1. La clase System: System.in, System.out y System.err 10
   2.2. Entrada desde teclado: Uso de `Scanner` y `BufferedReader` 11
   2.3. Salida a pantalla: Métodos `print`, `println` y `printf` 14
3. Operaciones con Ficheros (Lectura y Escritura) 17
   3.1. Tipos de Ficheros: Texto vs. Binarios 17
   3.2. Ciclo de vida y Modos de Acceso 19
   3.2. Ciclo de vida y Modos de Acceso 21
   3.3. Lectura y Escritura de información 24
4. Persistencia de Objetos 28
   4.1. Concepto de Serialización: Guardar el estado de un objeto 28
   4.2. La interfaz `Serializable` 29
   4.3. Flujos de objetos: Uso de `ObjectInputStream` y `ObjectOutputStream` 32
5. Gestión del sistema de ficheros 34
   5.1. Manipulación del entorno: la clase file (legado) y la api nio.2 (path, files) 34
   5.2. Gestión de estructura: Creación, verificación y eliminación 37
6. Interfaces Gráficas de Usuario (GUI) Simples 40
   6.1. Introducción a la programación dirigida por eventos 40
   6.2. Concepto de Evento: Fuente, objeto evento y oyente 41

1. Fundamentos de los Flujos (Streams)

1.1. Concepto de Stream: El flujo de datos en Java

Un **Stream** es, esencialmente, una **tubería de datos**. Es un canal lógico que conecta tu programa con una fuente de datos o un destino.

Imagina que tu programa Java es un depósito de agua. Para llenarlo (leer datos) o vaciarlo (escribir datos), necesitas conectar tuberías. A Java no le importa de dónde viene el agua (si de un río, un camión cisterna o un pozo), solo le importa que la tubería esté conectada y el agua fluya.

Características clave de los Streams:

1. **Abstracción Universal:** Lo genial de los Streams es que abstraen el dispositivo físico. Tu código para leer datos será casi idéntico si estás leyendo un archivo de texto en el disco duro (`hard drive`), recibiendo datos de internet (`network socket`) o leyendo lo que el usuario escribe en el teclado. Java oculta la complejidad del hardware bajo la alfombra del Stream.

2. **Unidireccionales:** A diferencia de una calle de doble sentido, los Streams en `java.io` son de sentido único.

   - **Input Stream (Flujo de Entrada):** Solo sirve para **leer** (traer datos hacia el programa).
   - **Output Stream (Flujo de Salida):** Solo sirve para **escribir** (enviar datos fuera del programa).
   - *Si quieres leer y escribir en un archivo a la vez, necesitarás dos flujos separados.*

3. **Acceso Secuencial:** Los datos fluyen en orden, uno detrás de otro, como una fila de hormigas. Lees el primer byte, luego el segundo, y así sucesivamente. No puedes "saltar" aleatoriamente al final del flujo sin haber pasado por el medio (aunque existen clases especiales para acceso aleatorio que veremos más adelante).

Diferencia vital: Stream I/O vs. Stream Java 8

¡Cuidado con la confusión de nombres! En la Unidad 7 mencionamos los Streams de Java 8 (`java.util.stream`) para filtrar listas y colecciones. En esta Unidad 8 hablamos de los Streams de E/S (I/O) (`java.io`). Aunque comparten nombre, son conceptos distintos:

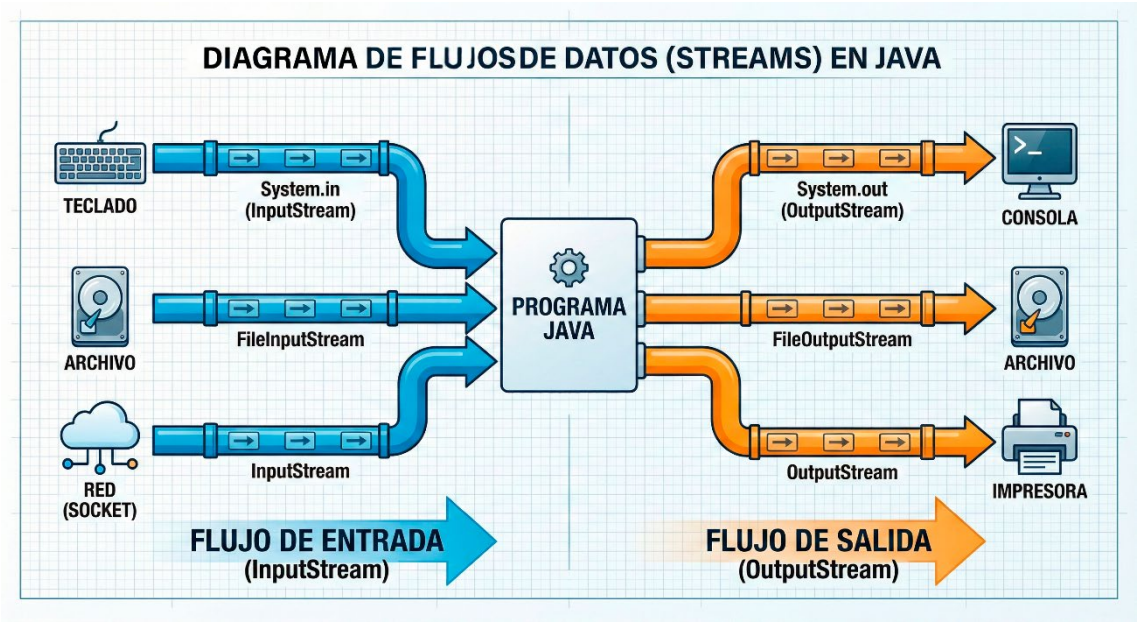
- **Java 8 Streams:** Procesan datos que ya están en memoria (RAM).
- **Java I/O Streams:** Mueven datos desde/hacia fuera del programa (Disco/Red).

El esquema mental: Fuente y Destino

Para entender cualquier operación de entrada/salida (I/O) en Java, siempre debes hacerte dos preguntas:

1. **¿Dónde está el dato? (La Fuente):** Puede ser un archivo, un array de bytes en memoria, una conexión TCP/IP, o el teclado.
2. **¿A dónde va el dato? (El Destino):** Puede ser un archivo, la pantalla (consola), una impresora, o enviarlo a otro ordenador.

El **Stream** es el puente que une ambos puntos.



1.2. Tipos de flujos: Bytes vs. Caracteres

En los inicios de Java (versión 1.0), solo existían los flujos de bytes. Pero los ingenieros de Sun Microsystems se dieron cuenta de que el mundo no habla solo en ASCII (inglés). Para soportar idiomas como el español (con sus ñ y tildes), chino o árabe, tuvieron que inventar una segunda categoría de flujos.

Así que hoy tenemos una **bifurcación crítica**:

1. Flujos de Bytes (Byte Streams)

Son los flujos de "bajo nivel". Manejan los datos en su forma más pura y cruda: **grupos de 8 bits (1 byte)**.

- **¿Qué transportan?** Información binaria exacta, sin modificar nada.
- **¿Para qué se usan?** Para todo lo que **NO** sea texto legible por humanos:
  - Imágenes (`.jpg`, `.png`).
  - Audio y vídeo (`.mp3`, `.mp4`).
  - Archivos ejecutables (`.exe`, `.class`).
  - Datos serializados de objetos.

- **Las Clases Base:** Todas las clases de esta familia heredan de:
  - `InputStream` (Entrada de bytes).
  - `OutputStream` (Salida de bytes).

Analogía: Imagina una mudanza. Los flujos de bytes mueven las cajas cerradas. No les importa si dentro hay libros o platos, solo mueven "cajas" (bytes) de un sitio a otro.

2. Flujos de Caracteres (Character Streams)

Son flujos especializados en **texto**. Java utiliza internamente el estándar **Unicode** (16 bits) para representar caracteres. Estos flujos están diseñados para traducir automáticamente los bytes crudos a caracteres legibles y viceversa.

- **¿Qué transportan?** Caracteres (char), gestionando automáticamente la **codificación** (encoding).
- **¿Para qué se usan?** Para leer o escribir información de texto:
    - Archivos de texto plano (`.txt`, `.java`, `.html`, `.xml`, `.json`).
- **Las Clases Base:** Todas las clases de esta familia heredan de:
    - `Reader` (Entrada de caracteres).
    - `Writer` (Salida de caracteres).

Analogía: Siguiendo con la mudanza, estos flujos son como un bibliotecario. Abren la caja, sacan el libro, leen el título y se aseguran de que esté en el idioma correcto. Entienden el contenido.

El Problema de la Codificación (Encoding)

¿Por qué es crítica esta diferencia? Si usas un flujo de bytes para leer un archivo de texto que contiene una "Ñ", el flujo verá un byte (o dos, dependiendo de la codificación del archivo) y te lo dará tal cual. Si tu programa no sabe interpretar eso, imprimirá un símbolo raro (como `Ã±` o `?`).

Los **Flujos de Caracteres (`Reader`/`Writer`)** actúan como **traductores inteligentes**.

1. Leen los bytes del disco.
2. Consultan la tabla de codificación (ej. UTF-8).
3. Combina esos bytes para formar el carácter correcto 'Ñ' en la memoria de Java.

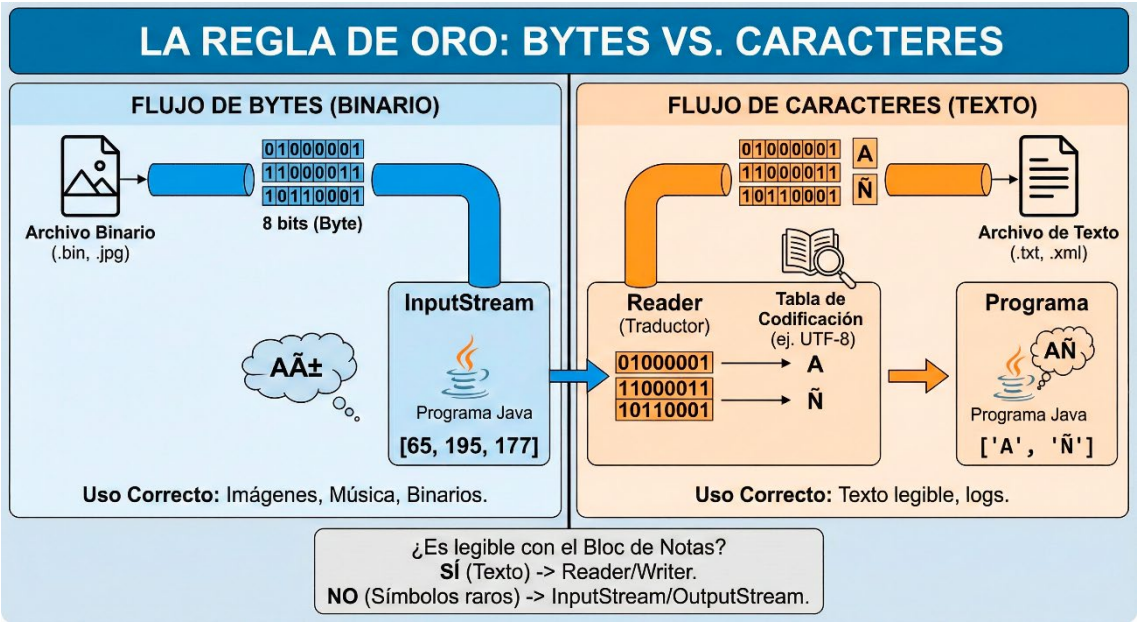
Tabla Resumen: La Regla de Oro

Tipo de Dato	Unidad de Info	Clases Abstractas (Padres)	Uso Correcto
Binario	Byte (8 bits)	`InputStream` / `OutputStream`	Imágenes, Música, Binarios.
Texto	Char (16 bits)	`Reader` / `Writer`	Texto (.txt, .xml, logs).

¿Cómo sé cuál usar?

La pregunta que debes hacerte siempre es: **¿Si abro este archivo con el Bloc de Notas, lo entiendo?**

- **SÍ** (es legible): Usa `Reader` y `Writer`.
- **NO** (veo símbolos raros y basura): Usa `InputStream` y `OutputStream`.



1.3. Jerarquía de Clases

1. La Rama de Bytes (Sufijo: `Stream`)

Como vimos, esta rama se usa para imágenes, audio, vídeos y cualquier dato binario. Las dos clases base son **abstractas**, lo que significa que no puedes hacer un `new InputStream()`. Tienes que usar una de sus hijas especializadas según **de dónde** vengan los datos.

A. `InputStream` (Entrada - Lectura)

Es la clase base para leer bytes de una fuente.

- **`FileInputStream`:** La más usada. Lee bytes de un archivo en el disco duro.
- **`ByteArrayInputStream`:** Lee bytes de un array en memoria.
- **`ObjectInputStream`:** Lee objetos Java serializados (veremos esto en el punto 8.4).
- **`BufferedInputStream`:** Un "envoltorio" que mejora la velocidad de lectura guardando datos en un búfer intermedio.

B. `OutputStream` (Salida - Escritura)

Es la clase base para escribir bytes hacia un destino.

- **`FileOutputStream`:** Escribe bytes en un archivo (creándolo o sobrescribiéndolo).
- **`ByteArrayOutputStream`:** Escribe bytes en un array en memoria (útil para crear buffers dinámicos).
- **`ObjectOutputStream`:** Convierte objetos a bytes para guardarlos.

Regla de nomenclatura: [Origen/Destino] + InputStream / OutputStream
Ejemplo: File + InputStream = Flujo de entrada desde archivo.

2. La Rama de Caracteres (Sufijos: `Reader` y `Writer`)

Esta rama se añadió más tarde para manejar texto internacional correctamente (Unicode).

A. `Reader` (Entrada - Lectura)

Es la clase base para leer caracteres.

- **`FileReader`:** Lee caracteres de un archivo de texto.
- **`StringReader`:** Lee caracteres de un `String` en memoria.
- **`BufferedReader`: ¡Fundamental!** Envuelve a otros Readers (como FileReader) para leer líneas completas de texto de golpe (método `readLine()`). Sin él, tendrías que leer carácter por carácter.
- **`InputStreamReader`: El Puente.** Convierte un flujo de bytes (`InputStream`) en un flujo de caracteres (`Reader`). Es el traductor que permite especificar la codificación (UTF-8, ISO-8859-1).

B. `Writer` (Salida - Escritura)

Es la clase base para escribir caracteres.

- **`FileWriter`:** Escribe texto en un archivo.
- **`PrintWriter`:** Muy popular porque tiene los métodos `print()` y `println()` que ya conoces de la consola, pero para escribir en archivos.
- **`BufferedWriter`:** Escribe bloques grandes de texto de forma eficiente.

Tabla Resumen de Equivalencias

Para cada clase de bytes suele haber una equivalente de caracteres. Aquí tienes las parejas más comunes:

Origen / Destino	**Bytes (Binario)**	**Caracteres (Texto)**
Clase Base	`InputStream` / `OutputStream`	`Reader` / `Writer`
Ficheros	`FileInputStream` / `FileOutputStream`	`FileReader` / `FileWriter`
Arrays / Strings	`ByteArrayInputStream`	`StringReader`
Eficiencia	`BufferedInputStream`	`BufferedReader`
Conversión	-	`InputStreamReader` / `OutputStreamWriter`

¿Por qué tantas clases? (El patrón Decorator)

Java usa un patrón de diseño llamado **Decorator** (Decorador). Esto significa que puedes "enchufar" flujos unos dentro de otros para combinar sus poderes.

Ejemplo: Quieres leer un archivo (`FileInputStream`) pero quieres hacerlo rápido (`BufferedInputStream`) y además quieres leer objetos Java (`ObjectInputStream`).

Java
```
// Enchufamos las tuberías una dentro de otra:
FileInputStream archivo = new FileInputStream("datos.dat");
BufferedInputStream buffer = new BufferedInputStream(archivo);
ObjectInputStream lectorObjetos = new ObjectInputStream(buffer);
```

Es como construir una tubería compleja con piezas de Lego.

1.4. Utilización de flujos: Patrones básicos y decoradores

En Java, rara vez usarás un solo objeto de flujo de forma aislada. La filosofía de `java.io` es la de **"construir tuberías"**. Usarás una clase para conectarte al archivo y otra clase (u otras dos) para leer cómodamente y rápido.

1. El Patrón Decorator (Las Muñecas Rusas)

Java utiliza el **Patrón Decorator**. Esto significa que "envolvemos" un flujo básico dentro de otro más avanzado para añadirle superpoderes.

Distinguimos dos tipos de flujos:

1. **Flujos de Nodo (Low-Level):** Se conectan directamente a la fuente (el "grifo" en la pared). Saben cómo hablar con el disco duro o la red.

   - *Ejemplos:* `FileInputStream`, `FileReader`.
   - *Problema:* Son "tontos" y lentos. Solo saben leer bytes o caracteres de uno en uno.

2. **Flujos de Proceso (High-Level / Decoradores):** No se conectan al disco. Se conectan a **otros flujos**. Su trabajo es procesar o mejorar los datos que pasan por ellos.

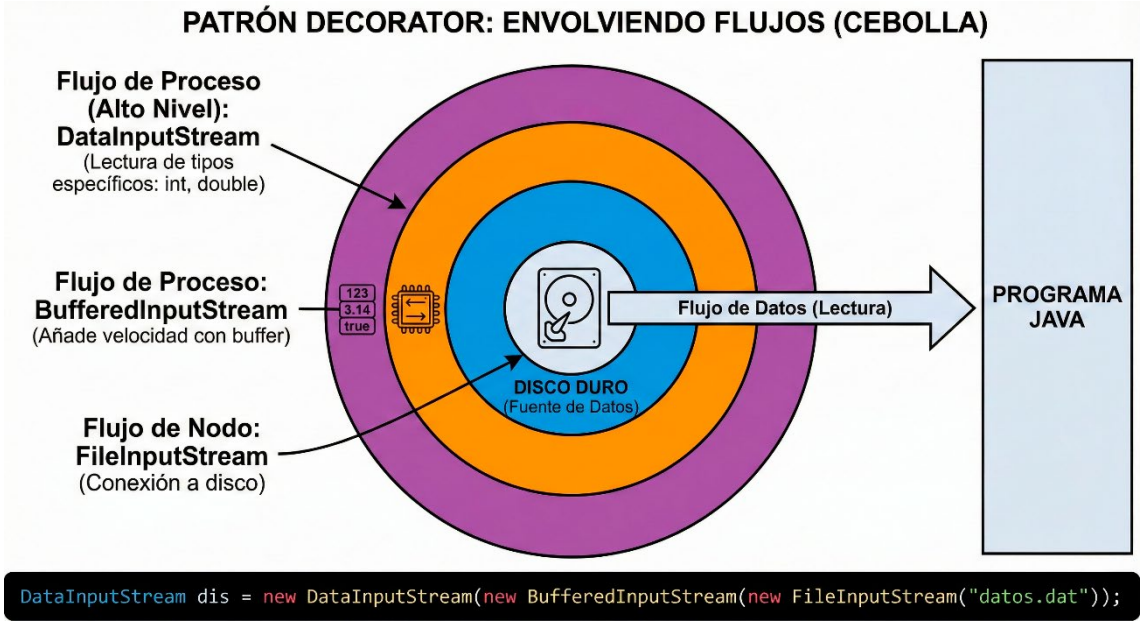
   - *Ejemplos:* `BufferedReader` (añade velocidad y lectura por líneas), `DataInputStream` (permite leer `int`, `boolean` directo).

La Sintaxis de "Cebolla"

Para crear un flujo potente, pasamos el flujo básico al constructor del flujo avanzado:

Java
```
// Versión paso a paso (Verbosa)
File archivo = new File("mi_novela.txt");
FileReader conector = new FileReader(archivo); // 1. Conecta al
disco
```

```
BufferedReader lector = new BufferedReader(conector); // 2. Añade
búfer y método readLine()

// Versión profesional (En una línea)
BufferedReader br = new BufferedReader(new
FileReader("mi_novela.txt"));
```



2. La importancia vital del Buffer (`Buffered...`)

De todos los decoradores, los **Buffered** (`BufferedReader`, `BufferedWriter`, `BufferedInputStream`) son los más importantes.

¿Por qué usarlos? El problema de la eficiencia. Acceder al disco duro es una operación físicamente lenta (el cabezal del disco tiene que moverse, o el SSD tiene que acceder a la celda).

- **Sin Buffer:** Si lees un archivo de 1000 caracteres con un `FileReader` normal, tu programa golpea la puerta del disco duro 1000 veces (una por cada carácter). Es ineficiente.
- **Con Buffer:** El `BufferedReader` golpea la puerta del disco **una sola vez**, se trae un bloque grande de datos (ej. 8192 caracteres) y los guarda en la memoria RAM (el búfer). Cuando tu programa pide el siguiente carácter, el `BufferedReader` se lo da desde la RAM instantáneamente.

Analogía:

- **Sin Buffer:** Vas al supermercado a comprar huevos. Entras, compras un huevo, vuelves a casa. Entras, compras otro huevo, vuelves a casa.
- **Con Buffer:** Vas al supermercado con un **carrito** (búfer). Llenas el carro de huevos de una vez y vuelves a casa. Luego los vas sacando del carrito según los necesitas.

3. Ejemplo Práctico: Lectura y Escritura

Veamos cómo se combinan las clases para leer y escribir texto, que es la operación más común.

A. Lectura eficiente (Texto)

Usaremos `FileReader` (nodo) envuelto en `BufferedReader` (decorador).
Java
```
try {
    // 1. Creamos el flujo compuesto
    BufferedReader br = new BufferedReader(new
FileReader("poema.txt"));

    String linea;
    // 2. Leemos línea a línea (gracias al BufferedReader)
    // readLine() devuelve null cuando llega al final del archivo
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }

    // 3. Importante: Al cerrar el decorador, se cierra
automáticamente el flujo interno
    br.close();

} catch (IOException e) {
    System.out.println("Error leyendo el fichero: " +
e.getMessage());
}
```

B. Escritura cómoda (Texto)

Usaremos `FileWriter` (nodo) envuelto en `PrintWriter` (decorador).

- ¿Por qué `PrintWriter`? Porque tiene los métodos `println()` y `printf()` que tanto nos gustan. `FileWriter` solo sabe escribir Strings planos.

Java
```
try {
    // El segundo parámetro 'true' en FileWriter indica modo
"Append" (añadir al final sin borrar lo anterior)
    PrintWriter pw = new PrintWriter(new FileWriter("diario.txt",
true));

    pw.println("Querido diario:");
    pw.printf("Hoy he aprendido %d conceptos nuevos de Java.\n",
3);

    pw.close(); // ¡Vital! Si no cierras, los datos pueden quedarse
atrapados en el búfer y no escribirse.

} catch (IOException e) {
    e.printStackTrace();
}
```

Resumen del patrón

1. Identifica qué vas a leer/escribir (Binario o Texto).
2. Elige el **Nodo** (File, Array, Network).

3. Elige el **Decorador** que te dé las funciones que necesitas (Velocidad -> Buffer, Comodidad -> Print/Data).
4. Enchúfalos en el constructor.

2. Entrada y Salida Estándar

Todo sistema operativo (Windows, Linux, macOS) proporciona a cualquier programa que se ejecute **tres canales de comunicación** por defecto. No tienes que abrirlos ni cerrarlos; el sistema te los regala al iniciar el programa.

Java encapsula estos tres canales en la clase **`java.lang.System`**.

2.1. La clase System: System.in, System.out y System.err

La clase `System` no se puede instanciar (no puedes hacer `new System()`). Estos tres canales son atributos **estáticos** (`static`) y públicos, por lo que podemos acceder a ellos desde cualquier lugar de nuestro código.

1. `System.out` (Salida Estándar)

Es el canal por el que el programa "habla" normalmente.

- **Tipo de dato:** Es un objeto de tipo **`PrintStream`** (una clase hija de `OutputStream`).
- **Destino por defecto:** La consola o terminal (la pantalla).
- **Uso:** Mostrar información, resultados o menús al usuario.
- **Métodos clave:** `print()`, `println()` (imprime y salta línea), `printf()` (formateo).

Curiosidad: `PrintStream` es de los pocos flujos que **nunca lanzan IOException**. Por eso nunca has tenido que poner un `try-catch` para escribir un `System.out.println`.

2. `System.in` (Entrada Estándar)

Es el canal por el que el programa "escucha".

- **Tipo de dato:** Es un objeto de tipo **`InputStream`** (Flujo de Bytes crudos). **¡Ojo a esto!**
- **Origen por defecto:** El teclado.
- **El problema:** Al ser un `InputStream`, lee **bytes**, no caracteres. Si lees directamente de `System.in`, obtendrás el código ASCII numérico de la tecla pulsada (ej: `97` en vez de `'a'`).
- **Solución:** Por eso casi nunca usamos `System.in` "a pelo". Lo normal es envolverlo en un `Scanner` o un `BufferedReader` para que nos traduzca esos bytes a texto y números.

3. `System.err` (Salida de Error Estándar)

Es el canal reservado para reportar problemas.

- **Tipo de dato:** También es un **`PrintStream`**, igual que `System.out`.
- **Destino por defecto:** También es la consola.

- **¿Cuál es la diferencia?**

    1. **Semántica:** Separa el "resultado útil" de los "avisos de fallo".
    2. **Visualización:** Los IDEs (como Eclipse o IntelliJ) suelen pintar lo que sale por `System.err` en color **ROJO**, para que destaque sobre el texto normal.
    3. **Sincronización:** A veces el sistema operativo los procesa en momentos distintos, por lo que el texto rojo puede salir desordenado respecto al texto negro.

Ejemplo Práctico: Los 3 canales en acción

Java
```
import java.io.IOException;

public class CanalesEstandar {
    public static void main(String[] args) {

        // 1. Uso de System.out (Información normal)
        System.out.println("Programa iniciado correctamente.");

        try {
            // 2. Uso de System.in (Directo, sin Scanner - Poco
habitual)
            System.out.print("Escribe una letra y pulsa Enter: ");
            int codigoByte = System.in.read(); // Lee UN solo byte

            System.out.println("Has pulsado la tecla con código
ASCII: " + codigoByte);

        } catch (IOException e) {
            // 3. Uso de System.err (Para errores)
            System.err.println("¡Ocurrió un error leyendo el
teclado!");
            e.printStackTrace(); // Esto también imprime por
System.err internamente
        }
    }
}
```

Redireccionamiento (Bonus)

Como estos son flujos (`Streams`), se pueden redirigir. Podrías decirle a Java: "Quiero que `System.out` no salga por la pantalla, sino que se escriba directamente en un fichero de texto".

Esto se hace con `System.setOut(new PrintStream("log.txt"));`. Muy útil para guardar registros (logs) sin cambiar todo tu código.

2.2. Entrada desde teclado: Uso de `Scanner` y `BufferedReader`

1. La opción moderna y versátil: La clase `Scanner`

Introducida en Java 5 (`java.util.Scanner`), se convirtió rápidamente en la favorita para aprender y para utilida0des sencillas.

- **Filosofía:** "Dime qué quieres (un entero, una línea, un decimal) y yo busco el siguiente trozo de información que encaje y te lo doy convertido".
- **Funcionamiento:** Divide la entrada en **tokens** (fichas) usando separadores (por defecto, el espacio en blanco o el salto de línea).

Ventajas:

- Sintaxis sencilla.
- **Conversión automática:** Tiene métodos como `nextInt()`, `nextDouble()`, `nextBoolean()`. No tienes que convertir tú el texto.

El "Bug" clásico del Scanner: Uno de los errores más comunes en Java ocurre cuando mezclas la lectura de números con la lectura de texto.

1. Lees un número con `nextInt()`. El Scanner lee el número pero **deja el "Enter" (`\n`) en el búfer**.
2. Luego llamas a `nextLine()` para leer un nombre.
3. El `nextLine()` encuentra ese "Enter" olvidado, piensa que es una línea vacía y se salta la lectura.
    - **Solución:** Hacer un `nextLine()` extra "de limpieza" después de leer números.

Ejemplo de uso correcto:
Java
```
import java.util.Scanner;

public class EjemploScanner {
    public static void main(String[] args) {
        // 1. Envolvemos System.in
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce tu edad: ");
        int edad = sc.nextInt(); // Lee el número, deja el \n

        // ¡LIMPIEZA DEL BÚFER! (Vital)
        sc.nextLine();

        System.out.print("Introduce tu nombre completo: ");
        String nombre = sc.nextLine(); // Ahora sí espera a que
escribas

        System.out.println("Hola " + nombre + ", tienes " + edad +
" años.");

        sc.close(); // Buena práctica cerrar, aunque cerrará
System.in
    }
}
```

2. La opción clásica y eficiente: La clase `BufferedReader`

Antes de que existiera Scanner, esta era la única forma. Pertenece a `java.io`.

- **Filosofía:** "Yo leo líneas completas de texto (Strings) de forma muy rápida y eficiente. Si quieres números, conviértelos tú".
- **Estructura:** Requiere el patrón de "muñecas rusas" que vimos antes para conectar los bytes del teclado con los caracteres.

Ventajas:

- **Rendimiento:** Es mucho más rápido que Scanner para grandes volúmenes de datos.
- **Sincronizado:** Es *Thread-safe* (seguro para hilos), importante en aplicaciones complejas.
- **Sin el bug del salto de línea:** Como siempre lee líneas enteras, no deja "basura" en el búfer.

Inconvenientes:

- **Verbosidad:** Requiere más código y gestionar excepciones (`try-catch` con `IOException`).
- **Todo es String:** Tienes que usar `Integer.parseInt()` o `Double.parseDouble()` manualmente.

Ejemplo de uso:
Java
```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class EjemploBufferedReader {
    public static void main(String[] args) {
        // Construcción de la tubería:
        // System.in (Bytes) -> InputStreamReader (Conversor a
Char) -> BufferedReader (Líneas eficientes)
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));

        try {
            System.out.print("Introduce tu edad: ");
            String textoEdad = br.readLine(); // Siempre devuelve
String
            int edad = Integer.parseInt(textoEdad); // Conversión
manual

            System.out.print("Introduce tu nombre: ");
            String nombre = br.readLine();

            System.out.println("Hola " + nombre + ", tienes " +
edad + " años.");

        } catch (IOException e) {
            System.err.println("Error de entrada/salida.");
        } catch (NumberFormatException e) {
            System.err.println("¡Eso no era un número válido!");
        }
        // Nota: No solemos cerrar br sobre System.in porque
cerraríamos el teclado para siempre en el programa.
    }
```

```
}
```

Resumen: ¿Cuál elijo?

Característica	Scanner (java.util)	BufferedReader (java.io)
Facilidad de uso	Alta (Ideal principiantes)	Media (Más código)
Conversión de datos	Automática (`nextInt`)	Manual (`Integer.parseInt`)
Velocidad	Lento (hace mucho análisis)	**Muy Rápido**
Excepciones	No verificadas (Runtime)	Verificadas (`IOException`)
Uso recomendado	Aplicaciones de consola pequeñas, ejercicios de clase.	Leer archivos grandes, sockets de red, sistemas profesionales.

2.3. Salida a pantalla: Métodos `print`, `println` y `printf`

Como vimos anteriormente, `System.out` es un objeto de tipo `PrintStream`. Este objeto nos ofrece tres herramientas principales para mostrar texto en la consola.

1. Lo básico: `print()` vs `println()`

La diferencia entre estos dos es el **cursor**.

- **`System.out.print(texto)`**: Escribe el texto y **deja el cursor al final de la línea**. Lo siguiente que imprimas aparecerá pegado justo a continuación.
- **`System.out.println(texto)`**: Escribe el texto y añade un carácter invisible de "retorno de carro" y "salto de línea". El cursor baja al inicio de la siguiente línea.

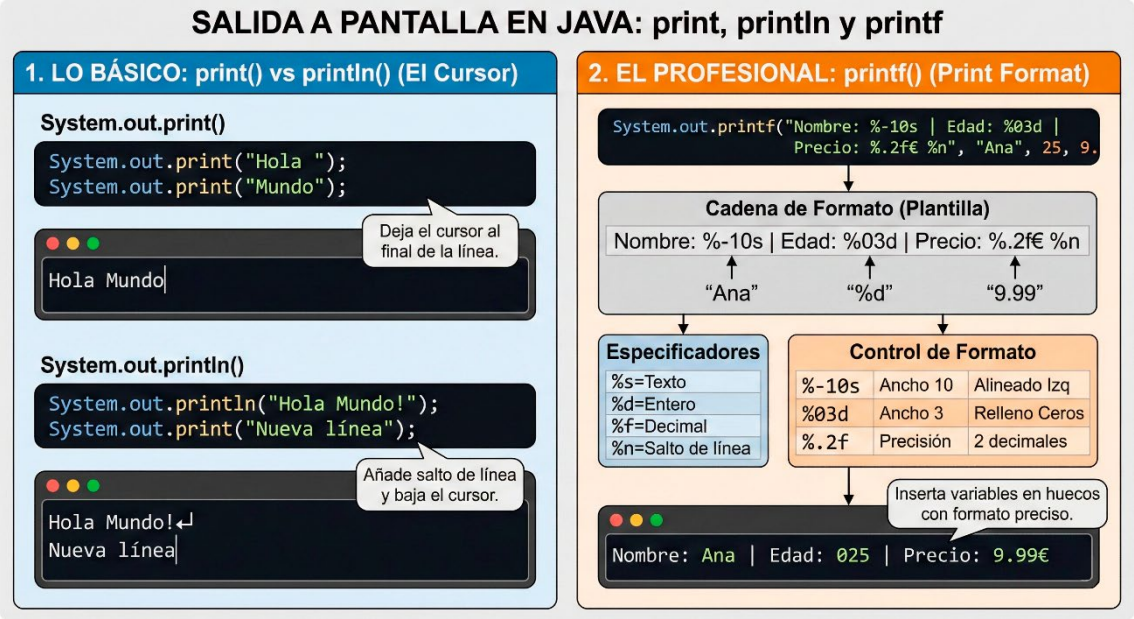
  Java
  ```
  System.out.print("Hola ");
  System.out.print("Mundo");
  System.out.println("!");
  System.out.print("Nueva línea");

  // Salida:
  // Hola Mundo!
  // Nueva línea
  ```

2. El profesional: `printf()` (Print Format)

Heredado del lenguaje C, `printf` es la herramienta definitiva para generar informes, tablas y datos numéricos limpios.

Su sintaxis es diferente: no concatenamos con `+`. En su lugar, usamos una **Cadena de Formato** con marcadores especiales (que empiezan por `%`) y luego pasamos las variables como argumentos separados por comas.

Sintaxis: `System.out.printf("Plantilla con %huecos", variable1, variable2...);`



Los Especificadores de Formato (Los "Huecos")

Cada vez que pones un `%`, Java espera un tipo de dato específico. Si te equivocas (ej: pones `%d` para un texto), el programa fallará (`IllegalFormatConversionException`).

Código	Tipo de Dato	Ejemplo
`%s`	String (Texto)	Nombres, palabras.
`%d`	Decimal Integer (Enteros)	`int`, `long`, `byte`. (¡Ojo! No es double).
`%f`	Floating point (Decimales)	`float`, `double`.
`%c`	Character	`char`.
`%b`	Boolean	`true` / `false`.
`%n`	Newline	Salto de línea (independiente del SO).

Controlando el Formato: Anchura y Precisión

La magia ocurre cuando añadimos instrucciones entre el `%` y la letra. La estructura es: `%[banderas][ancho][.precisión]tipo`

1. **Precisión (`.2`):** Fundamental para precios.

- `%.2f`: Redondea y muestra solo 2 decimales.

2. **Ancho (`10`):** Reserva un espacio fijo de caracteres. Vital para alinear columnas.

- `%10s`: Reserva 10 huecos y alinea a la **derecha** (rellena con espacios a la izquierda).
- `%-10s`: Reserva 10 huecos y alinea a la **izquierda** (rellena con espacios a la derecha).

3. **Banderas (`0`, `,`):**

- `%05d`: Rellena con ceros (ej: `00123`).
- `%,d`: Usa separadores de miles (ej: `1,000,000`).

Ejemplo Práctico: Generando un Ticket de Compra

Imagina intentar hacer esto solo con `println` y tabuladores... sería una pesadilla. Con `printf` es trivial.

Java
```java
public class TicketCompra {
    public static void main(String[] args) {
        String producto1 = "Pan";
        double precio1 = 0.85;
        String producto2 = "Leche Premium";
        double precio2 = 1.50;

        System.out.println("------------------------------");
        System.out.println("      TICKET DE COMPRA        ");
        System.out.println("------------------------------");

        // Cabecera: Alineado a izq (%-15s) y a der (%10s)
        System.out.printf("%-15s %10s %n", "PRODUCTO", "PRECIO");
        System.out.println("------------------------------");

        // Filas: Texto a izq, Flotante con 2 decimales a der
        System.out.printf("%-15s %10.2f€ %n", producto1, precio1);
        System.out.printf("%-15s %10.2f€ %n", producto2, precio2);

        System.out.println("------------------------------");

        double total = precio1 + precio2;
        // Total con formato moneda
        System.out.printf("%-15s %10.2f€ %n", "TOTAL", total);
    }
}
```

Salida en consola:

Plaintext
```text
------------------------------
      TICKET DE COMPRA
------------------------------
PRODUCTO            PRECIO
------------------------------
Pan                  0,85€
Leche Premium        1,50€
------------------------------
TOTAL                2,35€
```

(Nota: El separador decimal coma o punto depende de la configuración regional de tu ordenador).

3. Operaciones con Ficheros (Lectura y Escritura)

Un fichero no es más que una secuencia de bytes almacenada en un dispositivo de almacenamiento secundario (disco duro, SSD, USB). Sin embargo, la forma en que interpretamos esos bytes define dos mundos completamente distintos.

3.1. Tipos de Ficheros: Texto vs. Binarios

Aunque físicamente en el disco todo son unos y ceros, lógicamente los dividimos en dos categorías según **para quién** están diseñados.

1. Ficheros de Texto (Text Files)

Están diseñados para ser leídos por **humanos**.

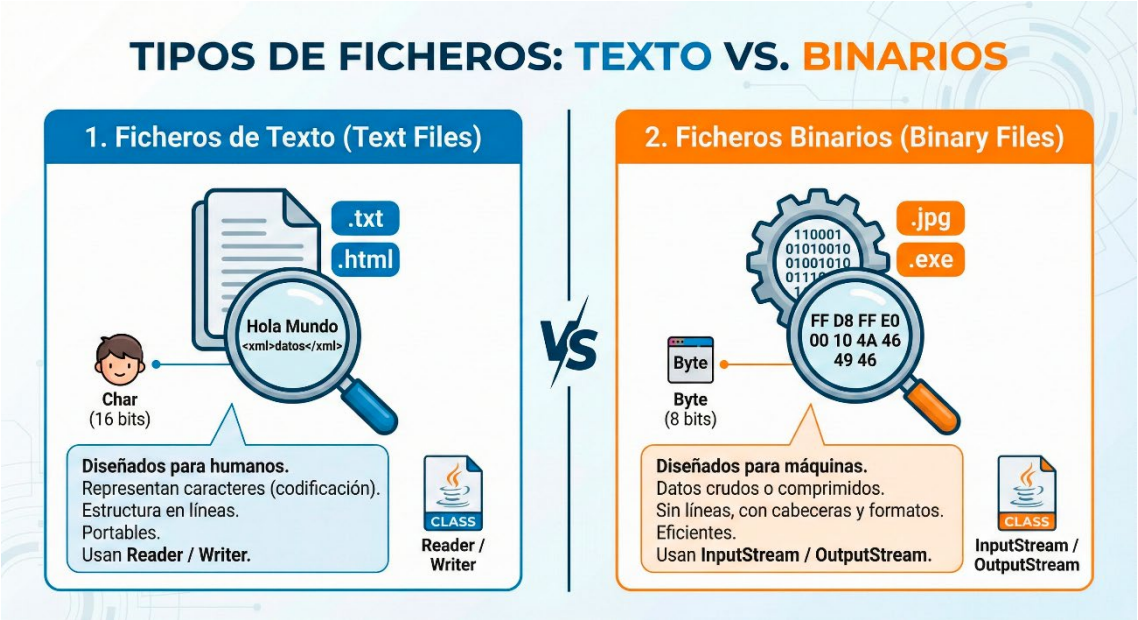
- **Representación:** Los bytes representan caracteres según una tabla de codificación (ASCII, UTF-8, ISO-8859-1).
- **Estructura:** Se organizan en **líneas**, separadas por caracteres especiales de salto de línea (`\n` o `\r\n`).
- **Independencia:** Son muy portables. Puedes abrir un `.java` o un `.html` creado en Windows con un ordenador Linux y leerlo sin problemas (salvo detalles menores de codificación).
- **Ejemplos:**
    - Código fuente (`.java`, `.py`).
    - Datos web (`.html`, `.json`, `.xml`).
    - Configuraciones (`.properties`, `.ini`).
    - Tablas simples (`.csv`).
- **Herramienta en Java:** Usaremos siempre las clases **`Reader`** y **`Writer`**.

2. Ficheros Binarios (Binary Files)

Están diseñados para ser leídos por **máquinas** (programas específicos).

- **Representación:** Los bytes representan datos en su formato interno o comprimido. No hay traducción a caracteres.
    - Un número entero `123456` en un fichero de texto ocupa 6 bytes (uno por dígito).
    - El mismo número en binario (`int`) ocupa 4 bytes fijos.

- **Estructura:** No tienen líneas. Tienen **formatos y cabeceras**. Por ejemplo, un archivo `.png` empieza con unos bytes específicos que dicen "Soy una imagen PNG", seguidos de bytes que definen el ancho, alto y los colores de los píxeles.
- **Eficiencia:** Son más compactos y rápidos de leer para el ordenador, porque no necesita "parsear" texto a números.
- **Ejemplos:**
    - Multimedia (Imágenes, Audio, Vídeo).
    - Ejecutables (`.exe`, `.class`).
    - Bases de datos (`.db`, `.sqlite`).
    - Archivos comprimidos (`.zip`).
- **Herramienta en Java:** Usaremos siempre las clases **`InputStream`** y **`OutputStream`**.



Concepto: Ficheros de Datos y Registros

Dentro de los ficheros binarios, existe una subcategoría muy importante en la programación clásica de gestión: los **Ficheros de Registros**.

Imagina que quieres guardar la información de tus Clientes sin usar una base de datos SQL.

- **Texto (`.csv`):** Guardarías: `1;Juan;2500.50\n`. Es fácil de ver, pero lento de procesar si tienes un millón de clientes.
- **Binario (Registros):** Guardas los bytes exactos de los tipos de datos de Java.
    - `int` (4 bytes) para el ID.
    - `String` (longitud fija o variable) para el Nombre.
    - `double` (8 bytes) para el Saldo.

Ventaja de los registros binarios: Como sabemos que cada cliente ocupa exactamente X bytes (registro de longitud fija), podemos usar **Acceso Aleatorio**. Podemos decirle a Java: "Salta directamente al byte 5000 y lee al cliente número 50", sin tener que leer los 49 anteriores. Esto es imposible en un archivo de texto variable.

Resumen Comparativo

Característica	Fichero de Texto	Fichero Binario
Legibilidad	Legible por humanos (Bloc de notas).	Ilegible (Símbolos raros).
Contenido	Caracteres y líneas.	Bytes crudos, tipos de datos, píxeles.
Tamaño	Generalmente mayor.	Más compacto y eficiente.
Corrupción	Si cambias un byte, se rompe una letra.	Si cambias un byte, el archivo puede quedar inutilizable.
Clases Java	`Reader` / `Writer`	`InputStream` / `OutputStream`

3.2. Ciclo de vida y Modos de Acceso

Trabajar con ficheros es como pedir prestado un libro de una biblioteca muy estricta (el Sistema Operativo). Tienes que pedirlo (**Abrir**), leerlo o escribir en él (**Procesar**) y, obligatoriamente, devolverlo (**Cerrar**). Si no lo devuelves, la biblioteca te penaliza (bloqueo de recursos).

1. El Cierre: La regla sagrada (`close` vs. `try-with-resources`)

Cuando abres un fichero, el Sistema Operativo asigna un recurso llamado "file handle" (manejador de fichero). Los sistemas tienen un límite de cuántos ficheros puede tener abiertos un programa simultáneamente.

Si olvidas cerrar los ficheros, ocurrirán dos desastres:

1. **Bloqueo:** En Windows, no podrás borrar ni mover el archivo porque "está siendo usado por otro programa" (tu propio programa).
2. **Memory Leaks:** Tu aplicación consumirá memoria hasta colapsar (`Too many open files`).

A. La Forma Antigua (Manual y Peligrosa)

Antes de Java 7, cerrar un fichero era tedioso. Tenías que usar un bloque `finally` para asegurarte de que se cerraba incluso si había errores, y dentro del `finally`, poner *otro* try-catch.

Java
```
// CÓDIGO LEGADO (NO USAR)
FileReader fr = null;
try {
```

```
    fr = new FileReader("datos.txt");
    // ... leer ...
} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        if (fr != null) fr.close(); // Cerrar manualmente
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

B. La Forma Moderna (`try-with-resources`)

Java 7 introdujo la **Gestión Automática de Recursos**. Si declaras el flujo entre paréntesis justo después del `try`, Java se encarga de cerrarlo automáticamente al llegar a la llave de cierre `}`, pase lo que pase (éxito o excepción).

Java
```
// CÓDIGO PROFESIONAL
try (FileReader fr = new FileReader("datos.txt");
     BufferedReader br = new BufferedReader(fr)) { // Puedes
declarar varios separándolos por ;

    String linea = br.readLine();
    System.out.println(linea);

} catch (IOException e) {
    System.out.println("Error procesando el fichero");
}
// ¡MAGIA! Aquí 'br' y 'fr' ya están cerrados (br.close() se llamó
solo).
```

2. Modos de Acceso: Secuencial vs. Aleatorio

Dependiendo de cómo necesitemos movernos por el archivo, existen dos filosofías.

A. Acceso Secuencial (Sequential Access)

Es el comportamiento por defecto de todos los **Streams** (`FileInputStream`, `FileReader`, `Scanner`).

- **Funcionamiento:** Como una cinta de casete antigua. Empiezas en el byte 0, luego el 1, luego el 2...
- **Limitación:** No puedes leer el final del archivo sin haber leído todo lo anterior. No puedes "rebobinar" fácilmente.
- **Uso:** Ideal para leer documentos de texto, logs, o procesar datos completos.

B. Acceso Aleatorio (Random Access)

Es un comportamiento especial que permite saltar a cualquier punto del archivo instantáneamente.

- **Funcionamiento:** Como un disco de vinilo o un disco duro. Tienes una "aguja" (puntero de archivo) que puedes mover a voluntad.
- **Clase Clave:** `RandomAccessFile`.
  - No hereda de `InputStream` ni `OutputStream`. Es una clase independiente muy potente.
- **Métodos clave:**
  - `seek(long pos)`: Mueve el puntero al byte indicado.
  - `getFilePointer()`: Te dice en qué byte estás ahora mismo.
  - `length()`: Tamaño total del archivo.

Ejemplo de Acceso Aleatorio: Imagina un archivo de 1GB. Quieres leer solo el último byte.

- *Secuencial:* Tienes que leer 1GB entero para llegar al final.
- *Aleatorio:* Haces `seek(length - 1)` y lees. Instantáneo.

Java
```
try (RandomAccessFile raf = new RandomAccessFile("datos.bin", "r"))
{ // "r" = read only
    // Saltamos al byte 100
    raf.seek(100);
    int dato = raf.read(); // Leemos solo ese byte
    System.out.println("El byte en la posición 100 es: " + dato);
}
```

3. Modos de Escritura: Sobreescribir vs. Añadir (Append)

Cuando abres un fichero para escribir (`FileWriter` o `FileOutputStream`), tienes que decidir qué pasa con el contenido antiguo si el archivo ya existe.

1. **Modo Sobreescritura (Overwrite) - Por defecto:** Si el archivo existe, lo borra y empieza de cero. Si no tienes cuidado, puedes perder datos.

Java
```
// Si 'log.txt' existía, su contenido se pierde para siempre.
FileWriter fw = new FileWriter("log.txt");
```

2. **Modo Añadir (Append):** El contenido nuevo se agrega al final del archivo existente. Se activa pasando un true como segundo parámetro en el constructor.

Java
```
// Mantiene lo antiguo y escribe al final.
FileWriter fw = new FileWriter("log.txt", true);
```

3.2. Ciclo de vida y Modos de Acceso

Trabajar con ficheros es como pedir prestado un libro de una biblioteca muy estricta (el Sistema Operativo). Tienes que pedirlo (**Abrir**), leerlo o escribir en él (**Procesar**) y, obligatoriamente, devolverlo (**Cerrar**). Si no lo devuelves, la biblioteca te penaliza (bloqueo de recursos).

1. El Cierre: La regla sagrada (`close` vs. `try-with-resources`)

Cuando abres un fichero, el Sistema Operativo asigna un recurso llamado "file handle" (manejador de fichero). Los sistemas tienen un límite de cuántos ficheros puede tener abiertos un programa simultáneamente.

Si olvidas cerrar los ficheros, ocurrirán dos desastres:

1. **Bloqueo:** En Windows, no podrás borrar ni mover el archivo porque "está siendo usado por otro programa" (tu propio programa).
2. **Memory Leaks:** Tu aplicación consumirá memoria hasta colapsar (`Too many open files`).

A. La Forma Antigua (Manual y Peligrosa)

Antes de Java 7, cerrar un fichero era tedioso. Tenías que usar un bloque `finally` para asegurarte de que se cerraba incluso si había errores, y dentro del `finally`, poner *otro* try-catch.

Java
```
// CÓDIGO LEGADO (NO USAR)
FileReader fr = null;
try {
    fr = new FileReader("datos.txt");
    // ... leer ...
} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        if (fr != null) fr.close(); // Cerrar manualmente
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

B. La Forma Moderna (`try-with-resources`)

Java 7 introdujo la **Gestión Automática de Recursos**. Si declaras el flujo entre paréntesis justo después del `try`, Java se encarga de cerrarlo automáticamente al llegar a la llave de cierre `}`, pase lo que pase (éxito o excepción).

Java
```
// CÓDIGO PROFESIONAL
try (FileReader fr = new FileReader("datos.txt");
     BufferedReader br = new BufferedReader(fr)) { // Puedes
declarar varios separándolos por ;

    String linea = br.readLine();
    System.out.println(linea);

} catch (IOException e) {
    System.out.println("Error procesando el fichero");
}
// ¡MAGIA! Aquí 'br' y 'fr' ya están cerrados (br.close() se llamó
solo).
```

2. Modos de Acceso: Secuencial vs. Aleatorio

Dependiendo de cómo necesitemos movernos por el archivo, existen dos filosofías.

A. Acceso Secuencial (Sequential Access)

Es el comportamiento por defecto de todos los **Streams** (`FileInputStream`, `FileReader`, `Scanner`).

- **Funcionamiento:** Como una cinta de casete antigua. Empiezas en el byte 0, luego el 1, luego el 2...
- **Limitación:** No puedes leer el final del archivo sin haber leído todo lo anterior. No puedes "rebobinar" fácilmente.
- **Uso:** Ideal para leer documentos de texto, logs, o procesar datos completos.

B. Acceso Aleatorio (Random Access)

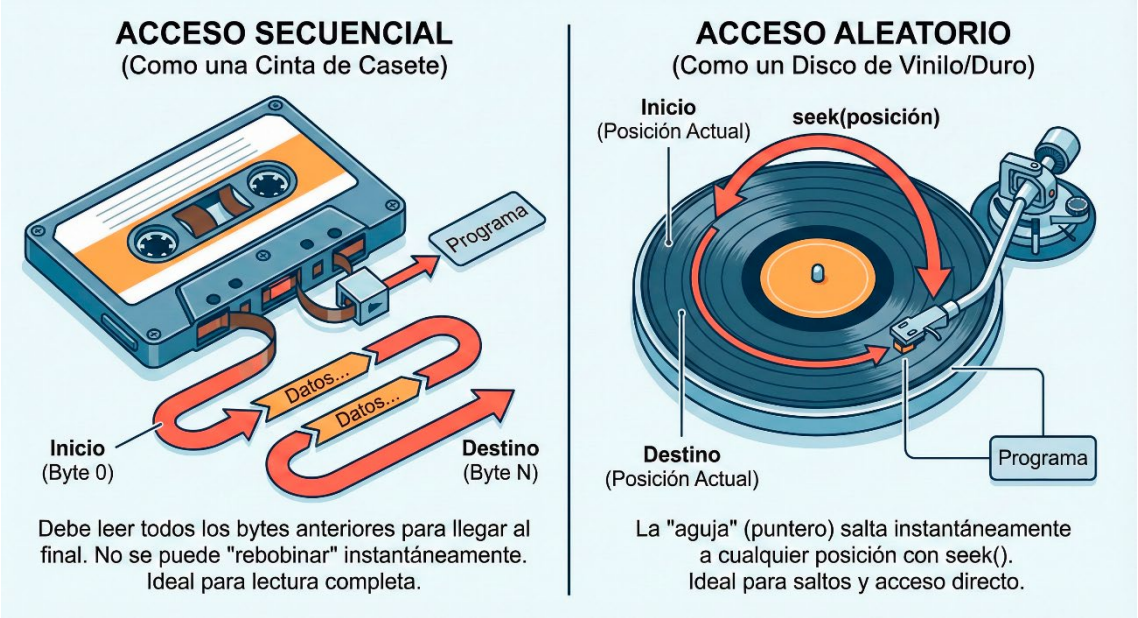
Es un comportamiento especial que permite saltar a cualquier punto del archivo instantáneamente.

- **Funcionamiento:** Como un disco de vinilo o un disco duro. Tienes una "aguja" (puntero de archivo) que puedes mover a voluntad.
- **Clase Clave:** **`RandomAccessFile`**.
    - No hereda de `InputStream` ni `OutputStream`. Es una clase independiente muy potente.
- **Métodos clave:**
    - `seek(long pos)`: Mueve el puntero al byte indicado.
    - `getFilePointer()`: Te dice en qué byte estás ahora mismo.
    - `length()`: Tamaño total del archivo.

Ejemplo de Acceso Aleatorio: Imagina un archivo de 1GB. Quieres leer solo el último byte.

- *Secuencial:* Tienes que leer 1GB entero para llegar al final.
- *Aleatorio:* Haces `seek(length - 1)` y lees. Instantáneo.

Java
```java
try (RandomAccessFile raf = new RandomAccessFile("datos.bin", "r"))
{ // "r" = read only
    // Saltamos al byte 100
    raf.seek(100);
    int dato = raf.read(); // Leemos solo ese byte
    System.out.println("El byte en la posición 100 es: " + dato);
}
```



3. Modos de Escritura: Sobreescribir vs. Añadir (Append)

Cuando abres un fichero para escribir (`FileWriter` o `FileOutputStream`), tienes que decidir qué pasa con el contenido antiguo si el archivo ya existe.

1. **Modo Sobreescritura (Overwrite) - Por defecto:** Si el archivo existe, lo borra y empieza de cero. Si no tienes cuidado, puedes perder datos.

Java
```
// Si 'log.txt' existía, su contenido se pierde para siempre.
FileWriter fw = new FileWriter("log.txt");
```

2. **Modo Añadir (Append):** El contenido nuevo se agrega al final del archivo existente. Se activa pasando un true como segundo parámetro en el constructor.

Java
```
// Mantiene lo antiguo y escribe al final.
FileWriter fw = new FileWriter("log.txt", true);
```

3.3. Lectura y Escritura de información

Para trabajar eficientemente, aplicaremos el **Patrón Decorator** que vimos antes: usaremos una clase para conectar al archivo (el "tubo") y otra para procesar los datos (el "filtro").

1. Clases para Texto: `Reader` y `Writer`

Estas clases gestionan automáticamente la codificación de caracteres (UTF-8, etc.) y las líneas.

A. Escritura con `FileWriter` y `PrintWriter`

Aunque `FileWriter` puede escribir, es muy limitado (solo escribe caracteres sueltos o arrays). Lo profesional es envolverlo en un **`PrintWriter`**, que nos regala los métodos `println()` y `printf()` que ya conoces de la consola.

Java
```
import java.io.*;

public class EscribirTexto {
    public static void main(String[] args) {
        // try-with-resources para cierre automático
        // "notas.txt" es el archivo. 'true' indica modo APPEND (no
borrar lo anterior)
        try (PrintWriter pw = new PrintWriter(new
FileWriter("notas.txt", true))) {

            pw.println("--- Inicio de Sesión ---");
            pw.printf("Usuario: %s | Nivel: %d%n", "Admin", 5);
            pw.println("Operación completada con éxito.");

            System.out.println("Fichero escrito correctamente.");

        } catch (IOException e) {
            System.err.println("Error escribiendo: " +
e.getMessage());
        }
    }
}
```

B. Lectura con `FileReader` y `BufferedReader`

`FileReader` conecta con el disco, pero lee carácter a carácter (ineficiente). **`BufferedReader`** es obligatorio aquí para leer líneas completas con `readLine()`.

Java
```
import java.io.*;

public class LeerTexto {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new
FileReader("notas.txt"))) {

            String linea;
            // El bucle estándar de lectura en Java:
            // Leemos una línea, la guardamos en 'linea', y
comprobamos que no sea null
            while ((linea = br.readLine()) != null) {
                System.out.println("Leído: " + linea);
            }

        } catch (FileNotFoundException e) {
            System.err.println("El archivo no existe.");
        } catch (IOException e) {
            System.err.println("Error de lectura.");
        }
    }
}
```

2. Clases para Binarios: `InputStream` y `OutputStream`

Aquí no hay líneas ni letras. Trabajamos con **tipos de datos primitivos** (int, double, boolean) o bytes crudos.

A. Bytes Crudos: `FileInputStream` y `FileOutputStream`

Se usan principalmente para **copiar archivos** (hacer un "Copiar y Pegar" de una imagen, por ejemplo). Leen byte a byte desde el origen y lo escriben en el destino.

Java
```
// Ejemplo conceptual de copiado de imagen
try (FileInputStream in = new FileInputStream("foto.jpg");
     FileOutputStream out = new FileOutputStream("copia_foto.jpg"))
{

    int byteLeido;
    // Leemos byte a byte (cuidado, esto es lento sin un buffer
intermedio)
    while ((byteLeido = in.read()) != -1) {
        out.write(byteLeido);
    }
}
```

B. Datos Estructurados: `DataInputStream` y `DataOutputStream`

Imagina que quieres guardar la puntuación de un juego (`int`), el nombre del jugador (`String`) y si tiene vidas infinitas (`boolean`).

- Si lo escribes como texto es fácil.
- Si lo escribes en binario, necesitas alguien que sepa convertir un `int` en exactamente **4 bytes**. Ese es el trabajo de `DataOutputStream`.

Escritura de datos primitivos (`DataOutputStream`):

Java
```
import java.io.*;

public class EscribirBinario {
    public static void main(String[] args) {
        // Envolvemos: File -> Buffered (opcional pero recomendado)
-> Data
        try (DataOutputStream dos = new DataOutputStream(
                                    new BufferedOutputStream(
                                    new
FileOutputStream("partida.dat")))) {

            dos.writeInt(1500);        // Escribe 4 bytes
            dos.writeDouble(99.50);    // Escribe 8 bytes
            dos.writeBoolean(true);    // Escribe 1 byte
            dos.writeUTF("Jugador1"); // Escribe String en formato
binario especial
```

```
            System.out.println("Datos binarios guardados.");

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Lectura de datos primitivos (`DataInputStream`):

¡Regla de Oro! Debes leer los datos en el **mismo orden exacto** en que los escribiste. Si escribiste `int` -> `double`, y lees `double` -> `int`, obtendrás números basura.

Java
```
import java.io.*;

public class LeerBinario {
    public static void main(String[] args) {
        try (DataInputStream dis = new DataInputStream(
                                   new BufferedInputStream(
                                   new
FileInputStream("partida.dat")))) {

            // Leemos en ORDEN EXACTO
            int puntos = dis.readInt();
            double salud = dis.readDouble();
            boolean vivo = dis.readBoolean();
            String nombre = dis.readUTF();

            System.out.println("Jugador: " + nombre);
            System.out.println("Puntos: " + puntos);

        } catch (IOException e) { // Se lanza EOFException si
intentas leer más allá del final
            System.err.println("Error leyendo datos.");
        }
    }
}
```

Resumen de Clases a Usar

Objetivo	**Quiero Escribir**	**Quiero Leer**
Texto Humano (.txt, .csv)	**`PrintWriter`** (sobre `FileWriter`)	**`BufferedReader`** (sobre `FileReader`)
Copiar Ficheros (.jpg, .pdf)	`FileOutputStream`	`FileInputStream`
Datos Primitivos (int, double)	**`DataOutputStream`**	**`DataInputStream`**

4. Persistencia de Objetos

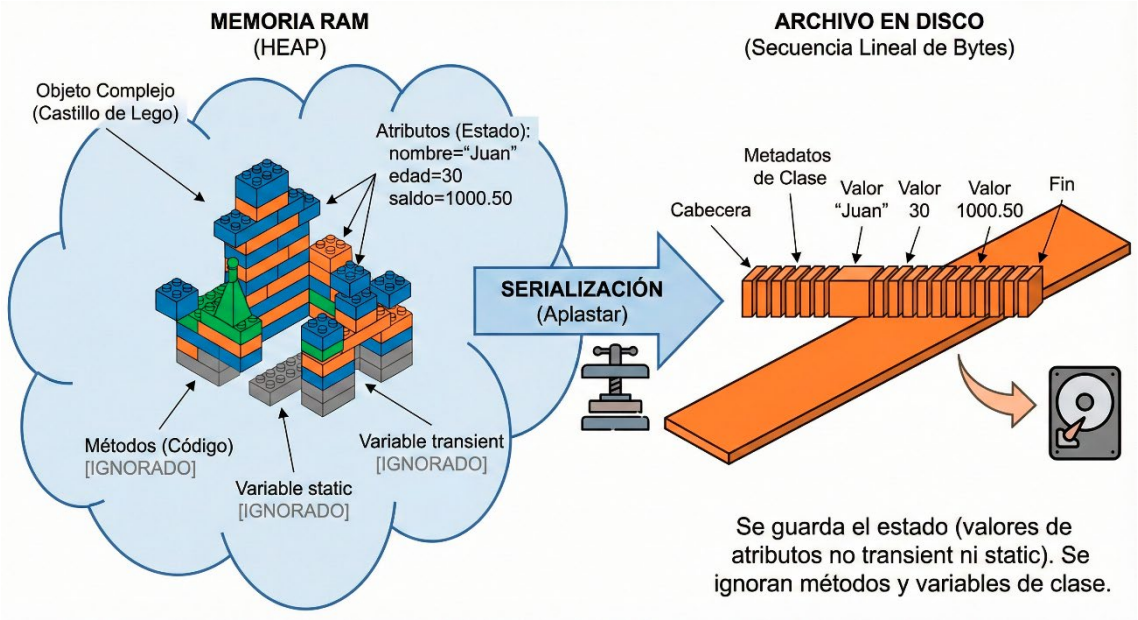
La persistencia es la capacidad de un objeto de sobrevivir más allá del proceso que lo creó. Si apagas el ordenador, el objeto sigue existiendo en el disco duro para ser recuperado después.

4.1. Concepto de Serialización: Guardar el estado de un objeto

La **Serialización** es el proceso de convertir un objeto en memoria (una estructura compleja y multidimensional en el Heap) en una **secuencia lineal de bytes**.

Imagina que tu objeto es un castillo de Lego montado.

- **En la RAM (Vivo):** El castillo está armado. Ocupa volumen y tiene estructura.
- **Serializar:** Es desmontar el castillo pieza por pieza y ponerlas en fila india dentro de una caja (el archivo).
- **Deserializar:** Es sacar las piezas de la caja y volver a montar el castillo exactamente como estaba antes.



¿Qué es "El Estado" de un objeto?

Cuando decimos que guardamos el "estado", nos referimos exclusivamente a **los valores de sus atributos de instancia** (las variables).

Al serializar, Java guarda:

1. **Los Datos:** El valor de `nombre`, `edad`, `saldo`, etc.
2. **Los Metadatos:** Qué tipo de clase es, la versión de la clase y la estructura de los atributos.
3. **El Grafo de Objetos:** Si tu objeto `Cliente` tiene dentro un objeto `Dirección`, Java es lo suficientemente listo para serializar al cliente **y** a la dirección automáticamente ("tira del hilo" y se lleva todo lo que esté conectado).

¿Qué NO se guarda?

- **Los Métodos:** El código no se guarda (ya está en el archivo `.class`).

- **Variables `static`:** Las variables estáticas pertenecen a la clase, no al objeto individual. No se serializan.
- **Variables `transient`:** Si marcas un atributo como `transient` (ej: una contraseña temporal), Java lo ignora y no lo guarda en el archivo.

¿Para qué sirve esto en el mundo real?

1. **Persistencia en Disco:** Guardar la partida de un videojuego (el objeto `Jugador` con todo su inventario) en un archivo `.sav`.
2. **Comunicación en Red:** Enviar un objeto desde un servidor en España a un cliente en Japón. Los cables de internet solo transportan bytes, no objetos Java. Serializamos para enviar y deserializamos al recibir.
3. **Caché:** Guardar objetos pesados en el disco para no tener que volver a calcularlos o pedirlos a la base de datos.

Analogía: La "Liofilización" (Comida Deshidratada)

Piensa en la serialización como **deshidratar comida**:

1. Tienes una sopa fresca (Objeto en RAM).
2. Le quitas el agua y la conviertes en polvo seco en un sobre (Serialización a Archivo). Es fácil de almacenar y enviar por correo.
3. Meses después, abres el sobre, añades agua caliente, y ¡magia! Vuelves a tener la sopa fresca (Deserialización).

El peligro: Si intentas "hidratar" el polvo usando una receta diferente (una versión de la clase distinta a la que creó el archivo), la sopa saldrá mal (Error de versión `InvalidClassException`).

4.2. La interfaz `Serializable`

Para que un objeto pueda ser convertido a bytes, Java necesita tu **permiso explícito**. No todos los objetos deben ser serializados (por ejemplo, un objeto que representa una conexión abierta a una base de datos o un hilo en ejecución no tiene sentido guardarlo en el disco).

Para dar ese permiso, utilizamos la interfaz `java.io.Serializable`.

1. ¿Qué hace esta interfaz? (Interfaz Marcadora)

Si abres el código fuente de Java y miras la interfaz `Serializable`, verás algo curioso: **¡Está vacía!** No tiene métodos.

Java
```
public interface Serializable {
    // No hay métodos aquí dentro.
}
```

A esto se le llama **Interfaz Marcadora (Marker Interface)**.

- **Función:** No obliga a implementar ningún código. Simplemente actúa como una **etiqueta** o un "sello de aprobación" que le pegas a tu clase.
- **Comportamiento:** Cuando intentas serializar un objeto, la JVM comprueba: *"¿Tiene este objeto el sello de Serializable?"*.
    - **SÍ:** Procede a convertirlo a bytes.
    - **NO:** Lanza inmediatamente una excepción `NotSerializableException` y detiene el proceso.

2. El `serialVersionUID`: Control de versiones

Imagina este escenario de terror:

1. Creas la clase `Usuario` y guardas (serializas) un millón de usuarios en el disco.
2. Un mes después, modificas la clase `Usuario` añadiendo un nuevo campo: `private String telefono;`.
3. Intentas leer (deserializar) los usuarios antiguos.

¿Qué ocurre? Java detectará que la clase en el disco (versión vieja) y la clase en el código (versión nueva) son diferentes. Lanzará una `InvalidClassException` y no podrás recuperar tus datos.

Para evitar esto, usamos el `serialVersionUID`. Es un número de identificación único para la versión de la clase.

Java
```
public class Usuario implements Serializable {
    // Es como el DNI de la clase
    private static final long serialVersionUID = 1L;

    // ... resto de la clase
}
```

- **Si lo defines manualmente (Recomendado):** Aunque cambies el código (añadas métodos o atributos), si mantienes el `serialVersionUID = 1L`, Java intentará cargar los datos antiguos "haciendo lo posible" (los campos nuevos quedarán en null, los viejos se cargarán).
- **Si NO lo defines:** Java calcula uno automáticamente basándose en el código de la clase. Al mínimo cambio que hagas (incluso un espacio), el ID cambia y los archivos antiguos se vuelven ilegibles.

Consejo Profesional: Siempre, SIEMPRE define el `serialVersionUID` manualmente. La mayoría de los IDEs (Eclipse, IntelliJ) te ofrecen generarlo con un clic cuando implementas `Serializable`.

3. La palabra clave `transient`

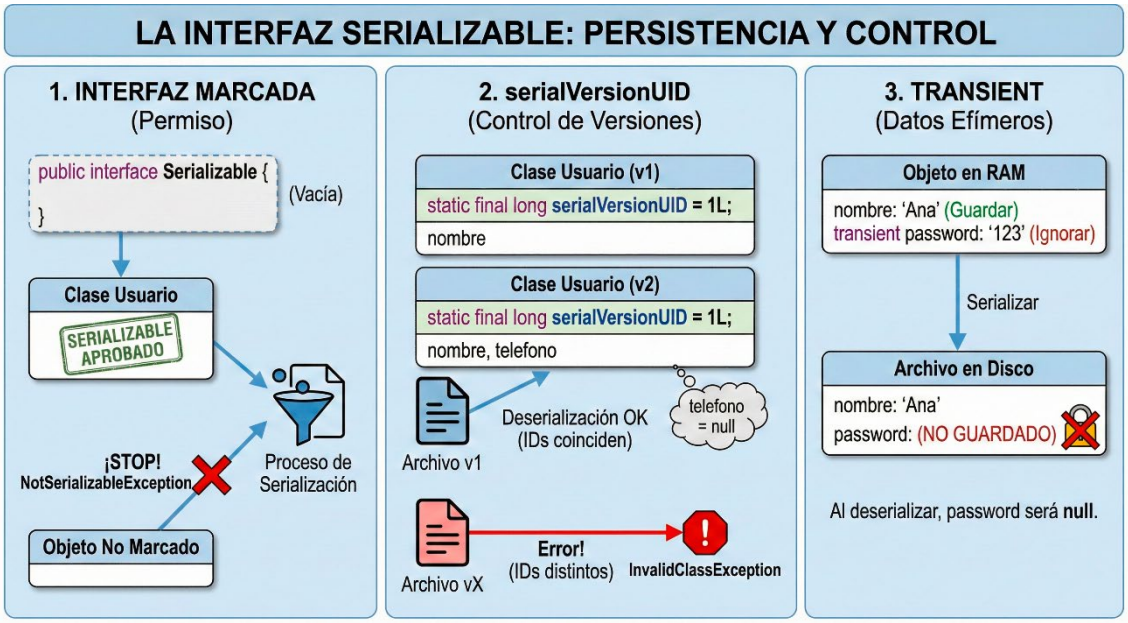
A veces, hay datos que **no** quieres guardar.

- **Información sensible:** Contraseñas, PINs bancarios.
- **Información calculada:** Un dato que se puede volver a calcular (ej: `edad`, si ya guardas la `fechaNacimiento`).
- **Referencias no serializables:** Conexiones, sockets.

Para excluir un atributo de la serialización, usamos la palabra reservada `transient`.

Java
```
public class Usuario implements Serializable {
    private static final long serialVersionUID = 1L;

    private String nombre;          // Se guardará
    private transient String password; // NO se guardará (será null
al leerlo)
    private int nivel;              // Se guardará
}
```

Al deserializar el objeto, el campo `password` aparecerá como `null` (o `0` / `false`), independientemente del valor que tuviera antes.



Ejemplo Completo de una Clase Serializable

Así es como se ve una clase lista para ser guardada en disco:

Java
```
import java.io.Serializable;

// 1. Implementamos la interfaz (El sello)
public class Videojuego implements Serializable {

    // 2. Definimos la versión para evitar problemas de
compatibilidad futura
    private static final long serialVersionUID = 1L;

    private String titulo;
    private int anio;

    // 3. Marcamos como transient lo que no queremos guardar
    // (Por ejemplo, una caché de gráficos que ocupa mucho y se
regenera al iniciar)
    private transient Object graficosTemporales;
```

```
    public Videojuego(String titulo, int anio) {
        this.titulo = titulo;
        this.anio = anio;
    }

    @Override
    public String toString() {
        return "Juego: " + titulo + " (" + anio + ")";
    }
}
```

4.3. Flujos de objetos: Uso de `ObjectInputStream` y `ObjectOutputStream`

Estas son las clases encargadas de la magia. Son **flujos de alto nivel (Decoradores)**, lo que significa que necesitan conectarse a un flujo de bajo nivel (como `FileOutputStream`) para saber *dónde* guardar los datos.

1. Escribir Objetos: `ObjectOutputStream` (Serialización)

Esta clase se encarga de coger un objeto, "aplastarlo" en bytes y enviarlo a la tubería.

- **Método clave:** `writeObject(Object obj)`.
- **Requisito:** El objeto que pases debe implementar `Serializable` (si no, lanzará `NotSerializableException`).

Ejemplo: Guardando el videojuego

Java
```
import java.io.*;

public class SerializarObjeto {
    public static void main(String[] args) {
        // Creamos el objeto en memoria
        Videojuego juego = new Videojuego("The Legend of Zelda",
1986);

        // Construimos la tubería: Fichero <- Buffer <-
ObjectOutput
        try (ObjectOutputStream oos = new ObjectOutputStream(
                                      new BufferedOutputStream(
                                      new
FileOutputStream("partida.save")))) {

            // ¡La magia ocurre aquí!
            oos.writeObject(juego);

            System.out.println("Objeto guardado en
'partida.save'");

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Más info: El archivo resultante (`partida.save`) será binario. Si intentas abrirlo con un editor de texto, verás símbolos raros, aunque quizás reconozcas alguna palabra suelta (los nombres de las clases o variables `string`).

2. Leer Objetos: `ObjectInputStream` (Deserialización)

Esta clase hace el proceso inverso: lee bytes, busca la definición de la clase en tu código (`.class`) y reconstruye el objeto en la memoria RAM (Heap).

- **Método clave:** `readObject()`.
- **Retorno:** Devuelve un tipo genérico `Object`.
- **El Casting:** Como devuelve `Object`, **siempre** tienes que hacer un **Casting explícito** al tipo de dato que esperas.

Ejemplo: Cargando el videojuego

Java
```
import java.io.*;

public class DeserializarObjeto {
    public static void main(String[] args) {

        try (ObjectInputStream ois = new ObjectInputStream(
                                     new BufferedInputStream(
                                     new
FileInputStream("partida.save")))) {

            // 1. Leemos (nos devuelve Object)
            Object objetoLeido = ois.readObject();

            // 2. Hacemos Casting (Convertimos Object ->
Videojuego)
            if (objetoLeido instanceof Videojuego) {
                Videojuego juegoCargado = (Videojuego) objetoLeido;
                System.out.println("Juego recuperado: " +
juegoCargado);
                // Imprimirá el toString() del objeto original
            }

        } catch (FileNotFoundException e) {
            System.err.println("¡No existe el archivo de
guardado!");
        } catch (IOException e) {
            System.err.println("Error de lectura: " +
e.getMessage());
        } catch (ClassNotFoundException e) {
            // Esto ocurre si tienes el archivo .save pero te falta
la clase Videojuego.java en tu proyecto
            System.err.println("El archivo contiene una clase que
no tengo definida.");
        }
    }
}
```

3. El Problema del `EOFException` (End of File)

Hay un detalle traicionero cuando guardas **varios objetos** en el mismo archivo (por ejemplo, una lista de 5 juegos uno detrás de otro).

Cuando lees un fichero normal byte a byte, al llegar al final te devuelve `-1` o `null`. Pero `readObject()` no funciona así. Si intentas leer un objeto y ya no quedan más en el archivo, lanza una excepción: `EOFException`.

Para leer todos los objetos de un archivo, el patrón estándar es un bucle infinito controlado por esta excepción:

Java
```
try {
    while (true) { // Bucle infinito intencional
        Videojuego v = (Videojuego) ois.readObject();
        System.out.println(v);
    }
} catch (EOFException e) {
    // Llegamos al final del archivo. Es una "excepción buena".
    System.out.println("--- Fin de la lectura de objetos ---");
}
```

Alternativa Profesional: En lugar de guardar 10 objetos sueltos, es mucho mejor meterlos todos en un `ArrayList<Videojuego>` y guardar **la lista entera** con un solo `writeObject(lista)`. Al leer, haces un solo `readObject()`, lo casteas a `ArrayList` y ya tienes todos tus datos sin pelear con el final del archivo.

5. Gestión del sistema de ficheros

5.1. Manipulación del entorno: la clase file (legado) y la api nio.2 (path, files)

Vamos a enfrentar a estas dos tecnologías.

1. La Vieja Escuela: La clase `java.io.File`

Desde Java 1.0, esta clase fue la reina. Un objeto `File` es una representación abstracta de una ruta de archivo o directorio.

- **Concepto:** Un objeto `File` **no** es el archivo físico. Es solo la **dirección** en memoria que apunta a él. Puedes crear un objeto `File` que apunte a `C:/unicornio.txt` aunque ese archivo no exista.
- **Problemas del Legado:**

    1. **Manejo de Errores Pobre:** Muchos métodos (como `delete()` o `mkdir()`) devuelven un `boolean` (`false` si falla). No te dicen *por qué* falló (¿falta de permisos? ¿carpeta llena? ¿bloqueado?).
    2. **Rendimiento:** Escalar directorios grandes puede ser lento.
    3. **Metadatos Limitados:** Difícil acceder a permisos avanzados o dueños del archivo.

Ejemplo de uso (`File`):
Java
```java
import java.io.File;

File foto = new File("vacaciones.jpg");

// Verificamos si existe (y si no es un directorio)
if (foto.exists() && !foto.isDirectory()) {
    System.out.println("Tamaño: " + foto.length() + " bytes");

    // Intentamos borrar
    boolean borrado = foto.delete();
    if (!borrado) {
        System.out.println("No se pudo borrar (y no sé por qué).");
    }
}
```

2. La Nueva Escuela: API NIO.2 (Java 7+)

Introducida en Java 7 bajo el paquete `java.nio.file` (Non-blocking I/O). Su objetivo fue solucionar todas las carencias de `File`. Separa la responsabilidad en dos piezas clave:

A. La Interfaz `Path` (La Dirección)

Reemplaza a la clase `File`. Representa, de forma mucho más potente, la ubicación de un elemento en el sistema de archivos. Es inmutable.

- Para crear uno, no usamos `new`, usamos la clase de ayuda `Paths` (o `Path.of` en Java 11+).

B. La Clase `Files` (El Operario)

Es una clase de utilidad estática que contiene **toda la lógica**. Si quieres hacer algo con un `Path` (copiar, mover, borrar, comprobar atributos), llamas a los métodos estáticos de `Files`.

- **Ventaja clave:** Lanza **Excepciones** (`IOException`) cuando algo falla. Te dirá exactamente "AccessDeniedException" o "NoSuchFileException".

Ejemplo de uso (NIO.2):
Java
```java
import java.nio.file.*;
import java.io.IOException;

// 1. Definimos la ruta (Path)
Path rutaFoto = Paths.get("vacaciones.jpg");

// 2. Usamos la clase Files para operar
if (Files.exists(rutaFoto)) {
    try {
        long tamano = Files.size(rutaFoto); // Método estático
        System.out.println("Tamaño: " + tamano);

        // Borrado con gestión de errores real
        Files.delete(rutaFoto);
        System.out.println("Borrado exitoso.");
```

```
    } catch (NoSuchFileException e) {
        System.err.println("El archivo no existe (alguien lo borró
antes).");
    } catch (DirectoryNotEmptyException e) {
        System.err.println("Es un directorio y tiene cosas
dentro.");
    } catch (IOException e) {
        System.err.println("Error de permisos u otro fallo I/O.");
    }
}
```

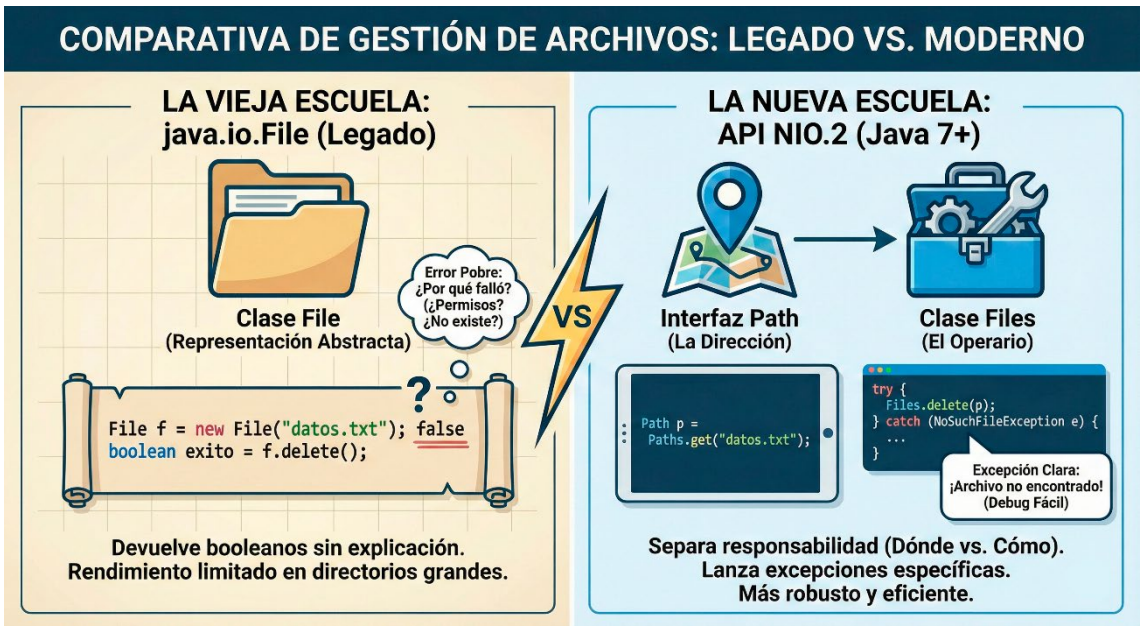


Tabla Comparativa: ¿Cuál usar?

Acción	Vieja Escuela (java.io.File)	Nueva Escuela (java.nio.file)
Referenciar ruta	`new File("ruta")`	`Paths.get("ruta")`
Comprobar existencia	`file.exists()`	`Files.exists(path)`
Borrar	`file.delete()` -> devuelve `false`	`Files.delete(path)` -> Lanza Excepción
Copiar archivo	*No nativo (había que hacer streams)*	`Files.copy(origen, destino)`
Mover/Renombrar	`file.renameTo(destino)`	`Files.move(origen, destino)`

Recomendación Profesional:

- Para **proyectos nuevos**: Usa siempre **NIO.2 (`Path` y `Files`)**. Es más robusto y claro.
- Para **proyectos antiguos**: Te encontrarás con `File` por todas partes. Debes saber leerlo, pero intenta migrar a `Path` cuando puedas (existe el método `file.toPath()` para convertir uno en otro).

5.2. Gestión de estructura: Creación, verificación y eliminación

Gestionar archivos no es solo leer y escribir. A menudo necesitarás que tu programa prepare su propio entorno: crear carpetas de configuración, comprobar si existen logs antiguos o limpiar archivos temporales al cerrar.

Todas las operaciones que veremos lanzan `IOException`, por lo que **siempre** deben ir dentro de un bloque `try-catch`.

1. Verificación: "Mira antes de saltar"

Antes de intentar leer o crear nada, debes preguntar al sistema de archivos.

- **¿Existe?** `Files.exists(path)` / `Files.notExists(path)`
- **¿Qué es?** `Files.isDirectory(path)` / `Files.isRegularFile(path)`
- **¿Puedo tocarlo?** `Files.isReadable(path)`, `Files.isWritable(path)`, `Files.isExecutable(path)`

Java
```
Path ruta = Paths.get("documentos/secreto.txt");

if (Files.exists(ruta) && Files.isRegularFile(ruta)) {
    System.out.println("El archivo existe y es real.");
} else {
    System.out.println("No existe o es un directorio.");
}
```

2. Creación: Archivos y Directorios

Aquí es donde la API de NIO.2 brilla por su precisión.

A. Crear Directorios (Carpetas)

Hay una distinción vital entre dos métodos:

1. **`Files.createDirectory(path)`**: Crea **solo** el último directorio de la ruta.
    - *Riesgo:* Si intentas crear `a/b/c` y la carpeta `a` no existe, lanzará una excepción (`NoSuchFileException`).
2. **`Files.createDirectories(path)`**: **(El Recomendado)** Crea toda la ruta necesaria.
    - *Ventaja:* Si `a` y `b` no existen, los crea automáticamente (como el comando `mkdir -p` en Linux). Si ya existen, no hace nada (no lanza error).

Java
```
Path rutaCarpetas = Paths.get("datos/usuarios/2023");
// Crea 'datos', 'usuarios' y '2023' si no existen
Files.createDirectories(rutaCarpetas);
```

B. Crear Archivos

Usamos **`Files.createFile(path)`**.

- **¡Cuidado!** Si el archivo ya existe, lanzará una `FileAlreadyExistsException`.
- *Uso típico:* Comprobar con `if (!Files.exists(path))` antes de crear.

3. Eliminación: Borrar con seguridad

Borrar archivos en Java es definitivo (no van a la papelera de reciclaje). Tenemos dos estrategias:

1. **`Files.delete(path)`**: El método estricto.
    - Si el archivo no existe -> Lanza `NoSuchFileException`.
    - Si es un directorio y **no está vacío** -> Lanza `DirectoryNotEmptyException`.
    - *Uso:* Cuando tu lógica de negocio exige que el archivo deba estar ahí para borrarlo.
2. **`Files.deleteIfExists(path)`**: El método suave.
    - Si existe, lo borra y devuelve `true`.
    - Si no existe, no hace nada y devuelve `false`.
    - *Uso:* Limpieza de archivos temporales (te da igual si existía o no, solo quieres asegurarte de que ya no esté).

Importante: Java **SE NIEGA** a borrar un directorio si tiene archivos dentro (para evitar catástrofes). Para borrar una carpeta llena, tienes que programar un recorrido recursivo que borre primero el contenido y luego la carpeta (o usar librerías externas como Apache Commons IO).

4. Copiar y Mover (Bonus Track)

Mover y copiar archivos también es trivial con la clase `Files`, pero hay un detalle clave: las **Opciones de Copia**.

Por defecto, si intentas copiar un archivo a un destino que ya existe, Java lanza una excepción (`FileAlreadyExistsException`). Para forzar la sobreescritura, debemos usar una constante especial: **`StandardCopyOption.REPLACE_EXISTING`**.

Java
```
Path origen = Paths.get("foto.jpg");
Path destino = Paths.get("backup/foto_copia.jpg");

// Copiar sobreescribiendo si ya existe
Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);

// Mover (Cortar y Pegar)
```

```
Files.move(origen, destino, StandardCopyOption.REPLACE_EXISTING);
```

Ejemplo Integral: El "Instalador"

Vamos a simular un pequeño script que prepara el entorno para una aplicación.

Java

```
import java.io.IOException;
import java.nio.file.*;

public class GestorArchivos {
    public static void main(String[] args) {
        Path directorioConfig = Paths.get("mi_app/config");
        Path archivoConfig =
directorioConfig.resolve("app.properties"); // Une rutas
inteligentemente

        try {
            // 1. Crear estructura de carpetas
            if (Files.notExists(directorioConfig)) {
                Files.createDirectories(directorioConfig);
                System.out.println("Directorios creados.");
            }

            // 2. Crear archivo vacío si no existe
            if (Files.notExists(archivoConfig)) {
                Files.createFile(archivoConfig);
                System.out.println("Archivo de configuración
creado.");
            } else {
                System.out.println("El archivo de configuración ya
existía.");
            }

            // 3. Crear y borrar un archivo temporal de prueba
            Path temp = Paths.get("mi_app/temp.lock");
            Files.createFile(temp);
            System.out.println("Archivo temporal creado...");

            // Simulamos proceso...

            boolean borrado = Files.deleteIfExists(temp);
            System.out.println("Limpieza temporal realizada: " +
borrado);

        } catch (IOException e) {
            // Captura genérica para errores de disco, permisos,
etc.
            System.err.println("Error en el sistema de archivos: "
+ e.getMessage());
            e.printStackTrace();
        }
    }
}
```

6. Interfaces Gráficas de Usuario (GUI) Simples

Hasta este momento, tus programas eran **dictadores**: ellos decidían el orden de todo.

1. "Pide el nombre".
2. "Espera a que escriba".
3. "Calcula".
4. "Imprime".
5. "Adiós".

El usuario no tenía libertad; si el programa pedía el nombre, el usuario no podía decidir cerrar la ventana o pulsar otro botón. Estaba "secuestrado" por el flujo secuencial del código.

En las **Interfaces Gráficas (GUI)**, esto cambia radicalmente. Aquí el usuario manda.

6.1. Introducción a la programación dirigida por eventos

La **Programación Dirigida por Eventos** (Event-Driven Programming) es un paradigma donde el flujo de ejecución del programa no está determinado por el orden de las líneas de código, sino por **sucesos externos** (eventos).

El cambio de mentalidad
Imagina la diferencia entre **leer un libro** y **trabajar de recepcionista**.

- **Programación Secuencial (Libro):** Empiezas en la página 1, luego la 2, luego la 3. Tienes el control total del progreso.
- **Programación por Eventos (Recepcionista):** Te sientas en el mostrador y **esperas**. No sabes qué pasará después.
    - Puede que suene el teléfono.
    - Puede que entre un cliente.
    - Puede que llegue un paquete.
    - Tú solo **reaccionas** cuando algo sucede.

¿Cómo funciona por dentro? (El Bucle de Eventos)

En una aplicación gráfica (como una calculadora), el método `main` no hace todo el trabajo y termina. Su función es diferente:

1. Dibuja la ventana (la interfaz).
2. Arranca un **Bucle de Eventos (Event Loop)** infinito.
3. El programa se queda en estado de **"Escucha" (Idle)**. No consume CPU, solo espera.

Cuando el usuario hace clic en un botón, el Sistema Operativo avisa a Java: *"¡Oye, han hecho clic en las coordenadas (50, 200)!"*. Java busca si hay algún trozo de código asociado a ese botón y, si lo hay, lo ejecuta. Al terminar, vuelve a dormirse (a esperar).

Diferencias Clave

Característica	Programación Secuencial (Consola)	Programación por Eventos (GUI)
¿Quién manda?	El Programador (el código).	El Usuario (o el sistema).
Flujo	Lineal (Inicio -> Fin).	No lineal (Saltos constantes).
Estado principal	Ejecutando instrucciones.	Esperando (Idle).
Predicción	Sabes exactamente qué pasará después.	No puedes predecir qué botón pulsará el usuario primero.

El Principio de Hollywood

Este paradigma se resume a menudo con la famosa frase de los castings de Hollywood:

> **"Don't call us, we'll call you"** (No nos llames, nosotros te llamaremos).

Tú no estás constantemente comprobando en un bucle `while` si el usuario ha pulsado una tecla (eso sería ineficiente). Tú defines el código y le dices al sistema: *"Cuando alguien pulse este botón, llámame y ejecuto esto"*.

6.2. Concepto de Evento: Fuente, objeto evento y oyente

Imagina una **Alarma de Incendios** en un edificio.

1. Alguien rompe el cristal y pulsa el botón (**Fuente**).
2. Se envía una señal eléctrica con la información de qué zona es (**Objeto Evento**).
3. La estación de bomberos recibe la señal y envía el camión (**Oyente**).

Vamos a traducir esto a Java (Swing/AWT).

1. La Fuente del Evento (Event Source)

Es el componente de la interfaz gráfica (GUI) donde ocurre la acción. Es el "gatillo".

- **¿Qué es?** Generalmente es un objeto visual como un botón (`JButton`), un campo de texto (`JTextField`), un menú o incluso la propia ventana.
- **Su función:** Detectar que el usuario ha interactuado con él.
- **Lo que NO sabe:** El botón no sabe qué hacer cuando lo pulsan. El botón `btnGuardar` solo sabe "ser pulsado", no sabe cómo guardar en la base de datos. Para eso necesita un Oyente.

En código: Es el objeto sobre el que llamamos al método `add...Listener()`.

2. El Objeto Evento (Event Object)

Es un paquete de información que se genera automáticamente cuando ocurre la acción. Es el "mensaje".

- **¿Qué es?** Es un objeto de una clase que hereda de `java.util.EventObject`.
- **Su función:** Transportar los detalles técnicos de lo que acaba de pasar.
- **Contenido típico:**
  - **`getSource()`:** ¿Quién generó el evento? (Referencia al botón pulsado).
  - **`getWhen()`:** ¿Cuándo ocurrió? (Timestamp).
  - **Detalles específicos:** Si es un evento de ratón (`MouseEvent`), dirá las coordenadas X e Y. Si es de teclado (`KeyEvent`), dirá qué tecla exacta se pulsó.

Tipos comunes:

- `ActionEvent`: Clics en botones, Enter en cajas de texto.
- `MouseEvent`: Movimientos, clics, arrastres del ratón.
- `KeyEvent`: Pulsaciones de teclado.
- `WindowEvent`: Cerrar, minimizar o abrir ventanas.

3. El Oyente (Listener)

Es el encargado de recibir el mensaje y ejecutar la lógica. Es el "cerebro".

- **¿Qué es?** En Java, un Listener es siempre una **Interfaz**.
- **Su función:** Define los métodos que se ejecutarán como respuesta. Como es una interfaz, nos obliga a implementar esos métodos.
- **El contrato:** La interfaz dice: *"Si quieres escuchar los clics de un botón, debes implementar la interfaz `ActionListener` y escribir el código dentro del método `actionPerformed`"*.

La Vinculación (Registration)

Esta es la parte crítica. Puedes tener un Botón (Fuente) y una Clase con lógica (Oyente), pero **si no los presentas, no pasará nada**.

Debes realizar el paso de **Registro**: El Oyente debe "suscribirse" a la Fuente.

Java
```
// Analogía conceptual
boton.addActionListener(miOyente);
```

Significa: *"Oye Botón, cuando te pulsen, avisa a `miOyente`"*. Si olvidas esta línea, el usuario pulsará el botón y Java ignorará el evento porque nadie está escuchando.

Resumen de Roles

Rol	**Componente Java (Ejemplo)**	**Analogía**	**Responsabilidad**
Fuente	`JButton`, `JTextField`	El botón de alarma	Generar el aviso.
Evento	`ActionEvent`	La señal eléctrica	Transportar los datos (qué, dónde, cuándo).
Oyente	`ActionListener`	Los bomberos	Recibir el aviso y **actuar** (ejecutar código).